

지식행동 사슬(KAC): 검증된 지식이 실행·결과·학습으로 전환되는 AX 실행 온톨로지

*The Knowledge-Action Chain (KAC): An AX Execution Ontology for
Turning Validated Knowledge into Action, Outcome, and Learning*

하우성 (Ryo Ha) · 박상훈 (Sang Hoon Park) · 정우빈 (Woo Bin Jeong)

sopia19910@gmail.com · pak3430@gmail.com · wjdnqs7@gmail.com

2026년 6월 23일

초록 (ABSTRACT)

AI 시대의 조직 운영에서 핵심 문제는 더 이상 "AI가 답을 만들 수 있는가"가 아니다. 생성형·에이전트형 AI는 이미 문서 작성·정보 탐색·분석·의사결정 보조·도구 실행·산출물 생성에 깊이 개입하고 있으나, AI가 만든 답이 곧바로 조직의 실행 역량이 되는 것은 아니다. 실제 문제는 검증된 지식이 특정 도메인 안에서 어떤 Skill로 도출되고, 그 Skill이 어떤 Runtime을 통해 실행 가능 상태가 되며, 실제 ActionEvent와 OutputObject·OutcomeObject를 만들고, Review와 Feedback을 거쳐 조직의 도메인 컨텍스트를 갱신하는가에 있다. 본 연구는 이 전체 구조를 **지식행동 사슬(Knowledge-Action Chain, KAC)**로 정의한다. KAC는 외부 지식사슬·스킬 도출 사슬

·SkillRuntime·ActionEvent·OutputObject·OutcomeObject·Review·Feedback·Context Update를 하나의 실행 온톨로지로 연결한다. 기존에 「지식사슬」이라 불려온 $Identity \rightarrow Goal \rightarrow Task \rightarrow Knowledge \rightarrow Method \rightarrow Skill$ 구조[R1]는 엄밀히 말해 지식사슬 전체가 아니라 특정 Identity·Entity가 왜 어떤 Skill을 필요로 하는지를 설명하는 **스킬 도출 사슬**이며, KAC는 그 이후의 실행·검증·학습 구조까지 담는다. 핵심 명제는 단순하다 — **지식사슬은 지식의 경로이고, 지식행동 사슬은 지식이 행동으로 검증되는 경로다**. 본 연구는 KAC의 9-튜플 정의, 선행 게이트 DCI[R6], SkillRuntime의 실행 계약, ActionEvent·OutputObject·OutcomeObject의 분리, 성립 조건 ValidKAC, 그리고 DCI·KCDI·RCI·AHCI·ARBI와의 운영 연결을 제시한다.

핵심어 지식행동 사슬, Knowledge-Action Chain, KAC, 스킬 도출 사슬, SkillRuntime, ActionEvent, OutcomeObject, 도메인 컨텍스트, DCI, KCDI, RCI, AHCI, ARBI, 조직 AX

1 문제 제기 — 지식은 왜 행동으로 닫혀야 하는가

AI 시대의 조직은 더 이상 "AI를 사용할 것인가"를 고민하는 단계에 있지 않다. 이미 많은 조직이 AI로 문서를 만들고, 회의를 요약하고, 고객 응대 초안을 작성하고, 전략 자료를 분석하고, 내부 지식을 검색하며, 여러 도구를 연결해 업무를 실행한다. 그러나 AI 사용량이 늘었다고 해서 조직 AX가 완성되는 것은 아니다. 조직 AX의 본질은 AI를 많이 쓰는 것이 아니라, AI가 조직의 목적·기준·역할·권한·검증·기록·책임·개선 구조 안에서 **반복 가능하게 작동하도록 만드는 것이다**.

공통 컨텍스트는 이 문제의 첫 번째 답이다. 공통 컨텍스트는 인간과 AI가 같은 기준으로 사고·작업하기 위해 공유하는 운영 기준이며, 목적·의미·상황·기준·역할·제한·출처·형식·피드백으로 구성된다[R4]. 그러나 공통 컨텍스트만으로는 충분하지 않다. 조직에는 고객응대·법무·교육·마케팅·개발·경영보고·ESG 등 여러 도메인이 있고, 각 도메인은 고유한 언어·지식·업무 흐름·위험 기준·검증 기준·승인 구조를 가진다. 따라서 공통 컨텍스트는 도메인별 실행 맥락으로 인스턴스화되어야 하고, 그 도메인 컨텍스트가 실제 기준 환경으로 사용 가능한지 검증되어야 한다 — 이것이 DCI(Domain Context Integrity)의 역할

이다[R6]. 한편 거버넌스 컨텍스트는 AI가 어떤 맥락에서 판단하고 어떤 권한으로 실행하며 어떤 기준으로 검증되고 누가 승인하며 어디에 기록되고 어떻게 책임지는가를 정하는 네트워크 운영 질서다[R4]. 이제 핵심 질문은 다음으로 바뀐다.

검증된 지식은 어떻게 실제 행동으로 전환되는가?
 도출된 Skill은 어떻게 실행 가능한 Runtime이 되는가?
 실행은 어떤 ActionEvent로 기록되는가?
 산출물은 어떤 OutputObject로 남고, 결과 변화는 어떤 OutcomeObject로 확인되는가?
 그 결과는 어떤 Review를 통과하고, 어떤 Feedback이 도메인 컨텍스트의 다음 버전에 반영되는가?

이 질문에 답하는 구조가 바로 지식행동 사슬(KAC)이다.

2 용어 정리 — 지식사슬·스킬 도출 사슬·지식행동 사슬

KAC를 정확히 이해하려면 먼저 세 구조를 분리해야 한다(표 1). 첫째, **외부 지식사슬(Knowledge Chain_ext)**은 지식이 생성·연결·추론·축적·전달·응용·재사용되는 흐름이다. 외부 문헌의 Knowledge Chain·Knowledge Value Chain·Knowledge Management Chain은 주로 지식의 획득·생성·저장·공유·적용·재사용 구조를 다루며[R7],[R8], 내부 레퍼런스도 지식사슬을 "개념에서 개념으로 허용된 간선만 밟아가는 풀이·추론의 경로"로 정의한다. 둘째, **스킬 도출 사슬(Skill Derivation Chain, SDC)**은 특정 Identity·Entity가 왜 어떤 Skill을 필요로 하는지를 도출하는 추론 경로 *Identity → Goal → Task → Knowledge → Method → Skill*이다[R1]. 셋째, **지식행동 사슬(KAC)**은 두 사슬을 기반으로 SkillRuntime·ActionEvent·OutputObject·OutcomeObject·Review·Feedback·Context Update 까지 포함하는 AX 실행 사슬이다.

표 1. 세 구조의 핵심 질문과 역할

구분	핵심 질문	역할
외부 지식사슬	지식은 어디서 오고 어떻게 연결·저장·공유·응용되는가?	지식 흐름
스킬 도출 사슬	특정 Identity/Entity는 왜 어떤 Skill을 필요로 하는가?	지식 → Skill 요구 도출
지식행동 사슬	도출된 Skill이 실제 Runtime·Action·Outcome으로 살아났는가?	AX 실행·검증·학습

이 구분이 중요한 이유는 명확하다. *Identity → ... → Skill*은 매우 중요한 구조지만 그 자체로는 아직 행동이 아니라 Skill 요구가 왜 정당한지를 설명하는 사슬이다. KAC는 그 이후 Skill이 실제 업무 세계에 진입하는 순간까지 다룬다.

3 KAC의 핵심 정의

KAC는 다음과 같이 정의된다 — **지식행동 사슬(KAC)**은 검증된 지식이 특정 도메인 컨텍스트 안에서 Skill로 도출되고, SkillRuntime을 통해 실행 가능 상태로 전환되며, 실제 ActionEvent·OutputObject·OutcomeObject·Review·Feedback을 거쳐 도메인 컨텍스트를 갱신하는 AX 실행·검증·학습 사슬이다. 더 짧게는 "지식을 실행 가능한 조직 능력으로 전환하는 사슬"이고, 가장 압축하면 "지식이 행동으로 검증되는 경로"다. 기본 경로는 그림 1과 같다.

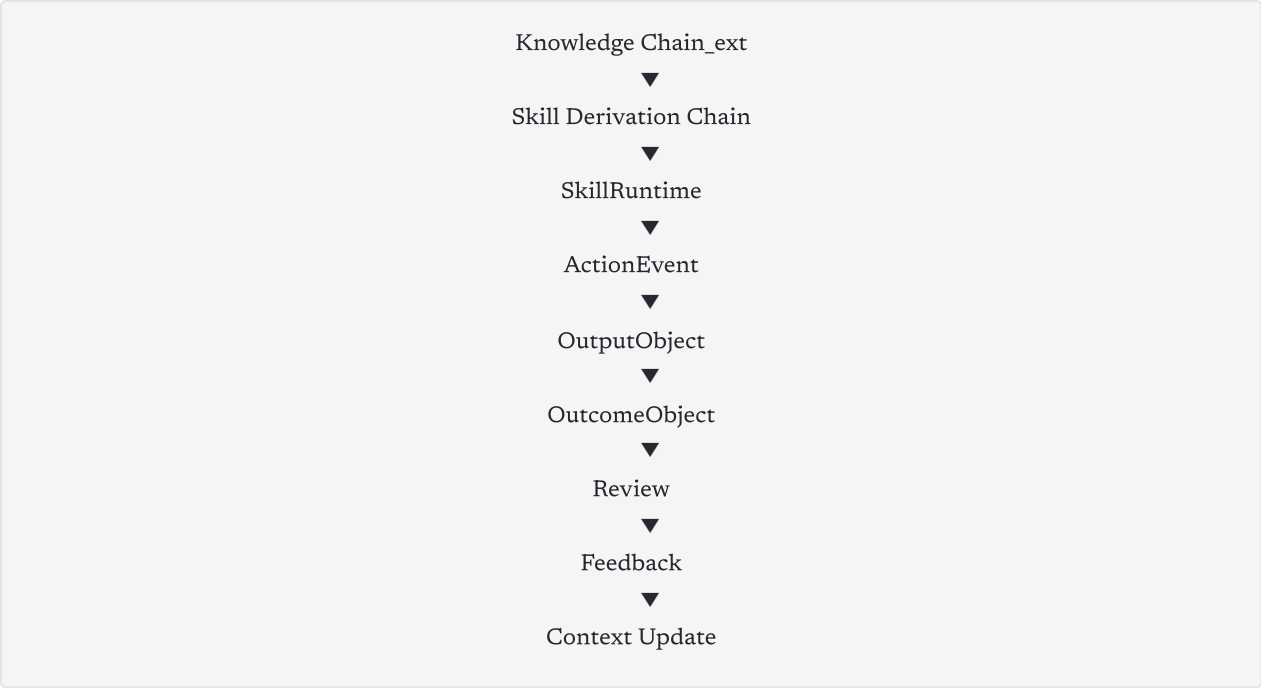


그림 1. KAC의 기본 경로 — 외부 지식사슬에서 출발해 스킬 도출·실행 계약·실행 사건·산출·결과·리뷰·피드백을 거쳐 도메인 컨텍스트 갱신으로 닫힌다.

객체형 수식으로는 9-튜플이다(표 2).

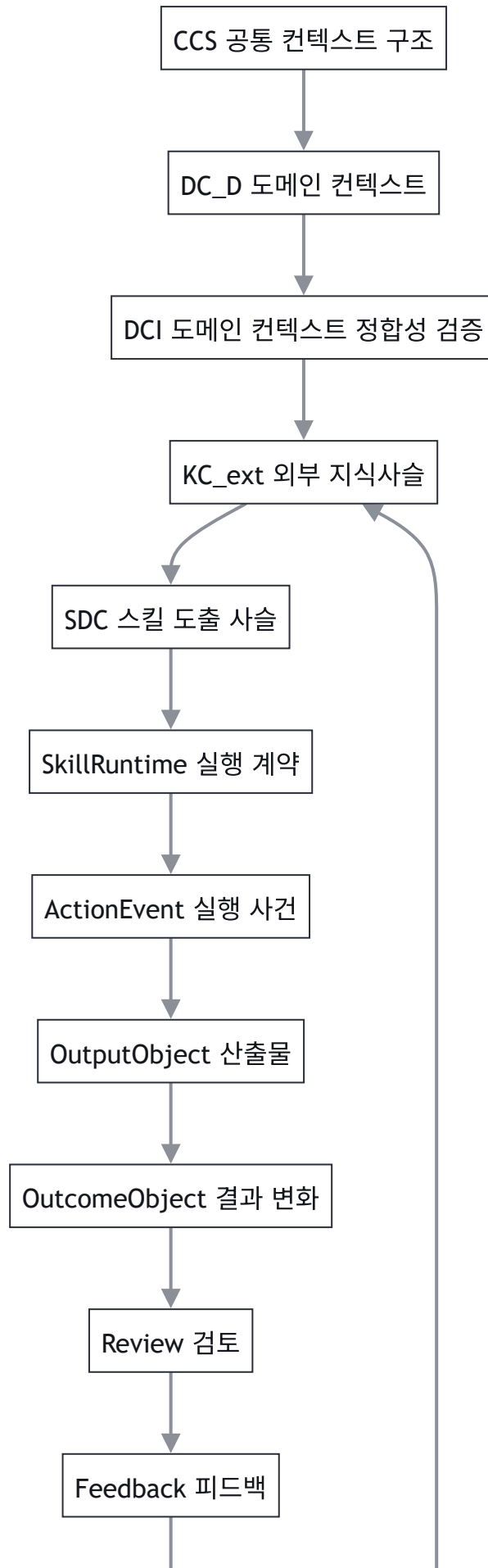
$$KAC = \langle KC^{ext}, SDC, R, A, P, \Omega, V, F, DC^D(t+1) \rangle$$

(1)

표 2. KAC의 9개 구성 요소

기호	이름	기호	이름 · 의미
KC_{ext}	External Knowledge Chain	Ω	OutcomeObject (결과 변화)
SDC	Skill Derivation Chain	V	Review (검토)
R	SkillRuntime (실행 계약)	F	Feedback (피드백)
A	ActionEvent (실행 사건)	$DC_D(t+1)$	갱신된 도메인 컨텍스트
P	OutputObject (산출물)		

이 9-튜플은 KAC를 개념 설명이 아니라 운영 객체들의 연결로 다루게 한다. 즉 KAC는 "AI가 답을 만들었다"가 아니라 "어떤 지식이 어떤 Skill로 도출되었고, 어떤 Runtime으로 실행되었으며, 어떤 ActionEvent와 산출물·결과를 만들었고, 그것이 어떤 검토와 피드백을 통해 조직 기준에 반영되었는가"를 묻는다.



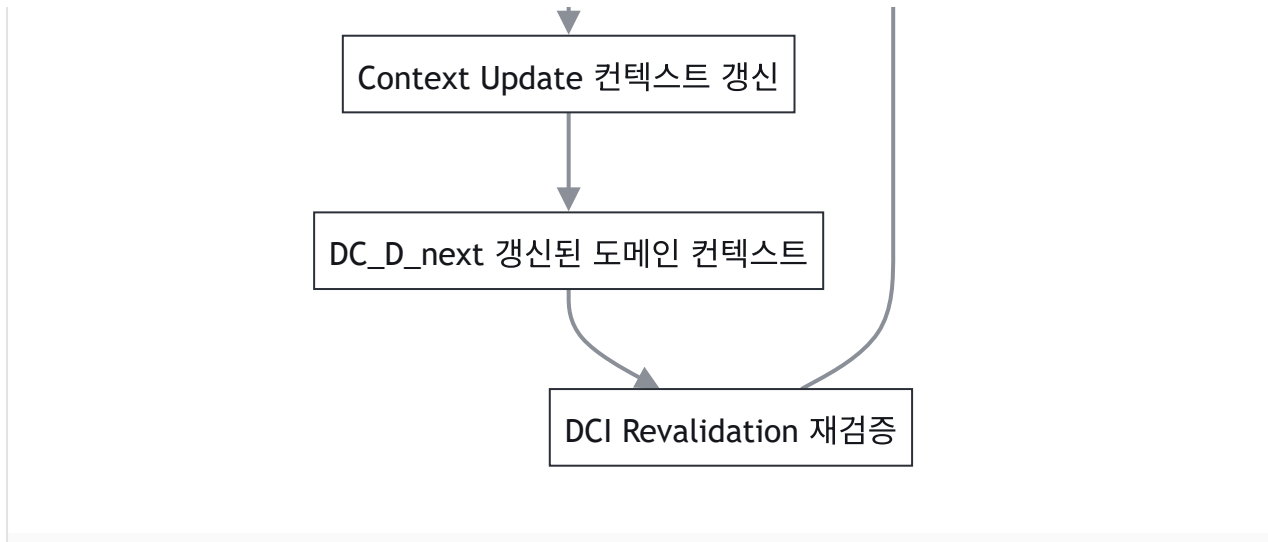


그림 2. KAC 기본 경로(순환) — 공통 컨텍스트 구조에서 도메인 컨텍스트·DCI를 거쳐 외부 지식사슬·스킬 도출·실행 계약·실행 사건·산출·결과·리뷰·피드백·컨텍스트 갱신으로 닫히고 DCI 재검증으로 순환한다.

4 선행 조건 — 도메인 컨텍스트와 DCI

KAC는 공중에 떠 있는 실행 사슬이 아니다. 성립하려면 먼저 특정 도메인 컨텍스트가 기준 환경으로 성립해야 한다. 공통 컨텍스트 구조(CCS)는 모든 도메인 컨텍스트가 따라야 하는 공통 운영 문법이고, 도메인 컨텍스트(DC_D)는 그 문법을 특정 도메인 D 에 적용한 실행 세계다[R6].

$$CCS = \langle \text{Purpose, Meaning, Situation, Criteria, Role, Limit, Source, Format, Feedback} \rangle \quad (2)$$

$$DC_D = \text{Instantiate}(CCS, D) = \langle D, CCS, I_D, G_D, T_D, K_D, M_D, S_D, E_D, \text{Src } D, \text{Val } D, \text{Gov } D, \text{Fb } D \rangle \quad (3)$$

DCI는 이 도메인 컨텍스트가 기준으로 사용 가능한지를 검증한다. DCI의 I 는 Index가 아니라 **Integrity**이며, 성과 점수가 아니라 정합성 검증이다 — "이 도메인 컨텍스트를 기준으로 사용해도 되는가?"이다. 전체 수식은 차단 게이트·완성 게이트·품질·위험 감쇠의 곱이다.

$$DCI_V(D) = G_B(D) \cdot G_C(D) \cdot Q_{DCI}(D) \cdot RA(D) \quad (4)$$

핵심은 점수보다 게이트가 먼저라는 점이다. 도메인 책임자 부재·운영 권한 부재·민감 자료 혼입·인간 최종 책임자 미명시·출처 위반·최소 거버넌스 부재 같은 치명적 결함은 높은 품질 점수로 보상될 수 없다. 따라서 KAC의 출발 조건은 명확하다 — **도메인 컨텍스트가 DCI를 통과해야만 ValidKAC가 시작된다.** 도메인 컨텍스트가 기준 환경으로 성립하지 않으면 그 위의 지식사슬·SkillRuntime·ActionEvent·Review·Feedback도 모두 불안정해진다.

5 스킬 도출 사슬 — KAC의 전반부

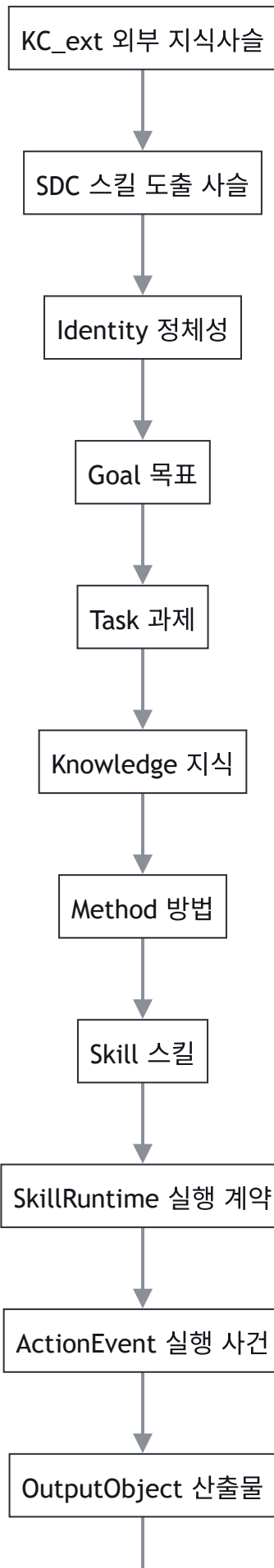
KAC의 전반부는 스킬 도출 사슬 $Identity \rightarrow Goal \rightarrow Task \rightarrow Knowledge \rightarrow Method \rightarrow Skill$ 이다. 여기서 중요한 것은 Identity와 Entity의 분리다. Identity는 존재 규정·역할·정체성·의미 구조이고, Entity는 그 Identity를 실제로 실현하는 주체다. 예컨대 *OxfordApplicant*는 Identity이고 *Sloane* 같은 실제 지원자는 Entity다. Identity는 내포(요구하는 Goal·Task·Knowledge·Method·Skill의 의미 구조)와 외연(그 Identity를 실현하는 Entity들의 집합)을 가진다[R1]. 이 분리는 KAC에서 결정적이다 — KAC는 "이 사람이 무엇을 해야 하는가"만 묻지 않고 "이 Identity를 실현하는 Entity가 어떤 Skill을 필요로 하고, 그 Skill이 어떤 Runtime과 Action으로 실행되었는가"를 묻기 때문이다.

스킬 도출 사슬은 Knowledge 노드에서 개발과 배포로 분해된다.

$$\begin{aligned}\pi^{dev} &= Identity \rightarrow Goal \rightarrow Task \rightarrow Knowledge, \quad \pi^{dep} \\ &= Knowledge \rightarrow Method \rightarrow Skill, \quad \pi = \pi^{dev} \oplus_K \pi^{dep}\end{aligned}\tag{5}$$

개발 구간은 정체성의 의미를 지식 요구로 확정하고, 배포 구간은 그 지식을 방법·스킬로 전환한다.

Knowledge는 개발의 산출물이자 배포의 입력이라는 이중 지위를 가진다[R1]. 그러나 Skill 노드가 생성되었다고 KAC가 완성되는 것은 아니다 — Skill은 아직 실행이 아니라 도출된 능력 요구이며, 그 요구가 실제 행동으로 이어지려면 실행 가능한 계약 구조가 필요하다. 이것이 SkillRuntime이다.



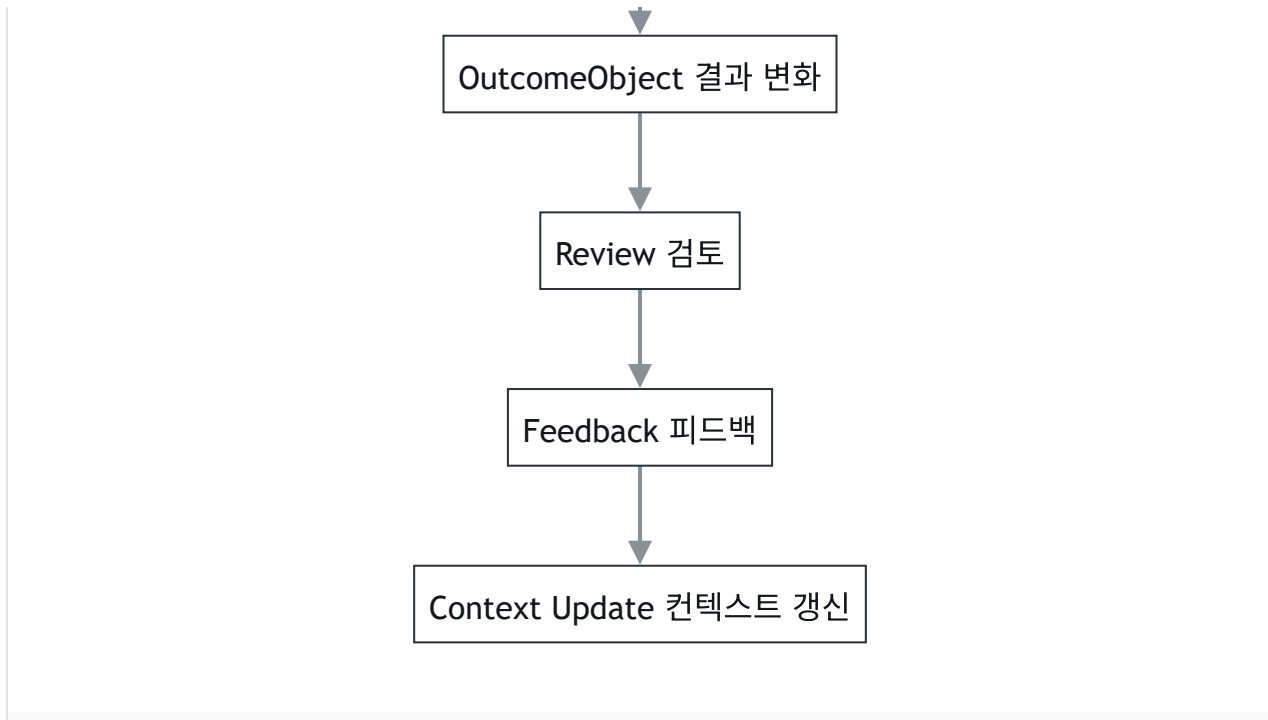


그림 3. SDC 내부와 KAC 후반부 — 외부 지식사슬에서 스킬 도출 사슬(Identity→Skill)이 펼쳐지고, 도출된 Skill이 SkillRuntime·ActionEvent·산출·결과·리뷰·피드백·컨텍스트 갱신으로 이어진다.

6 SkillRuntime — KAC의 핵심 전환점

KAC에서 가장 중요한 전환점은 *Skill* → *SkillRuntime*이다. Skill은 실행이 아니라 실행 요구다. "정책 기반 고객응대 스킬"이 도출되었다고 곧바로 고객응대가 실행되는 것은 아니며, 입력·출력·출처·도구·권한·검증·기록·승인·상태 관리·배포 정책이 명시되어야 한다. 따라서 — **SkillRuntime은 도출된 Skill을 실제 ActionEvent가 발생할 수 있는 실행 계약으로 컴파일한 구조다.** 단일 프롬프트나 단일 파일이 아니라 여러 실행 조건을 포함한 계약 그래프다(표 3).

$$\text{SkillRuntime} = \text{SkillInterface} + \text{InputContract} + \text{OutputContract} + \text{SourcePolicy} + \text{ToolPolicy} + \text{Guardrail} + \text{ValidationRule} + \text{TracePolicy} + \text{ApprovalPolicy} + \text{WorkflowSpec} + \text{InvocationContract} + \text{StateModel} + \text{ReleasePolicy} \quad (6)$$

표 3. SkillRuntime의 구성 요소와 역할

구성	역할	구성	역할
<i>SkillInterface</i>	외부와 만나는 접점	<i>TracePolicy</i>	실행 근거·로그 기록
<i>InputContract</i>	입력 스키마	<i>ApprovalPolicy</i>	인간 승인 조건
<i>OutputContract</i>	출력 스키마	<i>WorkflowSpec</i>	호출 순서·조건
<i>SourcePolicy</i>	사용 가능 자료	<i>InvocationContract</i>	호출 계약
<i>ToolPolicy</i>	호출 가능 도구	<i>StateModel</i>	실행 상태 관리
<i>Guardrail</i> · <i>ValidationRule</i>	위험·품질 제어	<i>ReleasePolicy</i>	배포·버전 정책

SkillRuntime이 없는 Skill은 실행되지 않은 능력 요구에 머문다. 반대로 SkillRuntime이 있으면 Skill은 실행 가능한 운영 객체가 된다. 이 점이 KAC를 단순한 지식 이론이 아니라 AX 실행 온톨로지로 만든다.

7 ActionEvent · OutputObject · OutcomeObject

SkillRuntime이 호출되면 실제 실행 사건 **ActionEvent**가 발생한다. ActionEvent는 지식이 실제 업무 세계에 진입한 순간이며, 단순한 "작업 수행"이 아니라 누가·언제·어떤 Runtime을·어떤 입력으로·어떤 권한과 출처와 도구 정책 아래 호출했는지를 포함하는 실행 기록이어야 한다.

$$\text{ActionEvent} = \langle \text{actor, runtime, input, source, tool, time, permission, trace, status} \rangle \quad (7)$$

ActionEvent의 결과로 **OutputObject**(산출물)가 생성된다 — 고객응대 답변 초안, 법무 검토 메모, 보고서, 요약문, 코드, 워크플로우 실행 결과, 평가표 등이다. 그러나 OutputObject와 **OutcomeObject**는 다르다. OutputObject는 만들어진 산출물이고, OutcomeObject는 그 산출물이 실제로 만든 결과 변화다.

$$\text{OutputObject} = \text{produced artifact}, \quad \text{OutcomeObject} = \text{verified change caused or supported by the output} \quad (8)$$

예컨대 AI가 고객 답변 초안을 만들었다면 그것은 OutputObject이고, 그 답변으로 문의가 해결되고 재문의가 줄고 정책 위반이 없었으며 만족도가 올랐다면 그것은 OutcomeObject다. 이 구분은 KAC에서 결정적이다 — 많은 AI 도입은 산출물 생성에서 멈추지만, KAC는 "AI가 무엇을 만들었는가?"가 아니라 "그것이 어떤 결과를 만들었고, 그 결과는 검증되었는가?"를 묻는다.

8 Review · Feedback · Context Update

KAC는 OutcomeObject에서 끝나지 않는다. OutcomeObject는 반드시 **Review**를 거쳐야 하며, 이때 검토는 단순한 품질 평가가 아니라 권한·책임·출처·근거·기준·표현·승인·기록·위험까지 포함한다. RCI는 이 Review를 발화·산출물 단위로 구체화하는 지표로, AI·증강 발화를 리뷰 가능한 운영 객체로 보고 권한·책임 위반은 차단 게이트로, 객체·근거·기록 미충족은 수정 게이트로 처리하며 통과 후보의 품질을 가중 기하평균과 위험 감쇠항으로 평가한다[R3]. Review를 통과한 결과는 **Feedback**으로 정리되는데, 단순 의견이 아니라 컨텍스트 변경 후보여야 한다.

$$\text{Feedback} = \langle \text{issue, evidence, affected_context, proposed_change, risk, owner, approval_status, rollback_plan} \rangle \quad (9)$$

Feedback이 승인되면 도메인 컨텍스트의 새 버전이 만들어지고, 새 도메인 컨텍스트는 다시 DCI를 통과해야 한다.

$$DC^D(t) + \text{ApprovedFeedback} \rightarrow DC^D(t+1) \rightarrow \text{DCI Revalidation} \quad (10)$$

이 구조가 KAC를 일회성 실행이 아니라 조직 학습 사슬로 만든다 — 지식에서 행동으로, 행동에서 결과로, 결과에서 검토로, 검토에서 피드백으로, 피드백에서 컨텍스트 갱신으로 닫힌다.

9 ValidKAC — KAC의 성립 조건

KAC는 경로가 있다고 자동 성립하지 않으며, 각 구간이 검증되어야 한다(표 4).

$$\text{ValidKAC} = \text{ValidKC} \wedge \text{ValidSDC} \wedge \text{RuntimeReady} \wedge \text{ValidAction} \wedge \text{ValidOutput} \wedge \text{ValidOutcome} \wedge \text{ReviewPassed} \wedge \text{FeedbackValid} \wedge \text{ContextRevalidated} \quad (11)$$

표 4. ValidKAC의 9개 성립 조건

조건	검증 질문
<i>ValidKC</i>	외부 지식사슬이 허용된 지식 관계와 출처를 따라 구성되었는가?
<i>ValidSDC</i>	Skill이 Identity 또는 Entity 기준에서 정당하게 도출되었는가?
<i>RuntimeReady</i>	Skill이 실행 가능한 Runtime 계약으로 컴파일되었는가?
<i>ValidAction</i>	ActionEvent가 권한·출처·도구·가드레일 안에서 수행되었는가?
<i>ValidOutput</i>	OutputObject가 스키마와 품질 기준을 만족하는가?
<i>ValidOutcome</i>	OutcomeObject가 목표 기준에서 실제 결과 변화를 보여주는가?
<i>ReviewPassed</i>	결과가 반영 가능하다는 검토를 통과했는가?
<i>FeedbackValid</i>	피드백이 근거·영향 범위·책임자·승인·rollback 계획을 갖추었는가?
<i>ContextRevalidated</i>	갱신된 도메인 컨텍스트가 다시 DCI를 통과했는가?

이 수식의 의미는 강하다 — 지식이 좋아도 Skill이 정당하게 도출되지 않으면, Skill이 정당해도 Runtime이 없으면, Action이 발생해도 Output이 기준 미달이면, Output이 있어도 Outcome이 없으면, Outcome이 있어도 Review를 통과하지 못하면, Feedback이 있어도 DCI 재검증을 통과하지 못하면 KAC는 성립하지 않는다. 따라서 KAC는 "지식이 실행되었다"는 느슨한 표현이 아니라, 지식·스킬·실행·산출·결과·검토·피드백·컨텍스트 갱신이 모두 성립한 경우에만 완성된다.

10 KAC와 운영 지표의 연결

KAC는 DCI·KCDI·RCI·AHCI·ARBI와 연결될 때 실제 운영 체계가 된다(표 5). DCI는 도메인 컨텍스트가 기준 환경으로 사용 가능한지 검증하는 선행 게이트로, KCDI를 보장하지는 않되 KCDI가 의미 있게 측정될 기준 환경을 제공한다[R6]. KCDI는 지식사슬 배포 성공도를 측정한다 — "도출된 Skill이 실제 실행 능력으로 살아났는가?"이며, 여섯 요소의 곱이다[R2].

$$\text{KCDI}(e, \pi, c) = \text{ValidDeployment}(\pi) \cdot \text{Adopt}(e, m, c) \cdot \text{Use}(e, m, c) \cdot \text{Improve}(e, s) \cdot \text{Trace}(\pi, e, c) \cdot \text{GovernanceFit}(c) \quad (12)$$

곱셈 구조는 하나의 치명적 결함이 전체 성공을 무너뜨림을 보장한다 — 논리적으로 정당하게 도출된 Skill도 현장에서 사용되지 않으면, 사용되어도 기록이 없거나 거버넌스를 위반하면 배포 성공으로 보기 어렵다. RCI는 KAC 후단의 Review를 발화·주장·응답·산출물 단위로 검증하고[R3], AHCI는 인간-봇

1:1 협력에서 인간이 목적 설정·맥락 제공·실행 지시·결과 해석·검증·책임 판단·기록·개선 능력을 유지·확장하는지를, **ARBI**는 AI가 개입한 협업에서 인간과 AI의 역할이 건강하게 분담·보완·검증·책임 있게 작동하는지를 평가한다[R5].

표 5. KAC 운영 체계의 다섯 지표

지표	역할
DCI	기준 환경 검증 (도메인 컨텍스트 정합성)
KCDI	Skill 현장 작동 측정 (배포 성공도)
RCI	산출물·발화 반영 가능성 검증
AHCI	인간-AI 증강 역량 측정
ARBI	협업 역할균형 측정

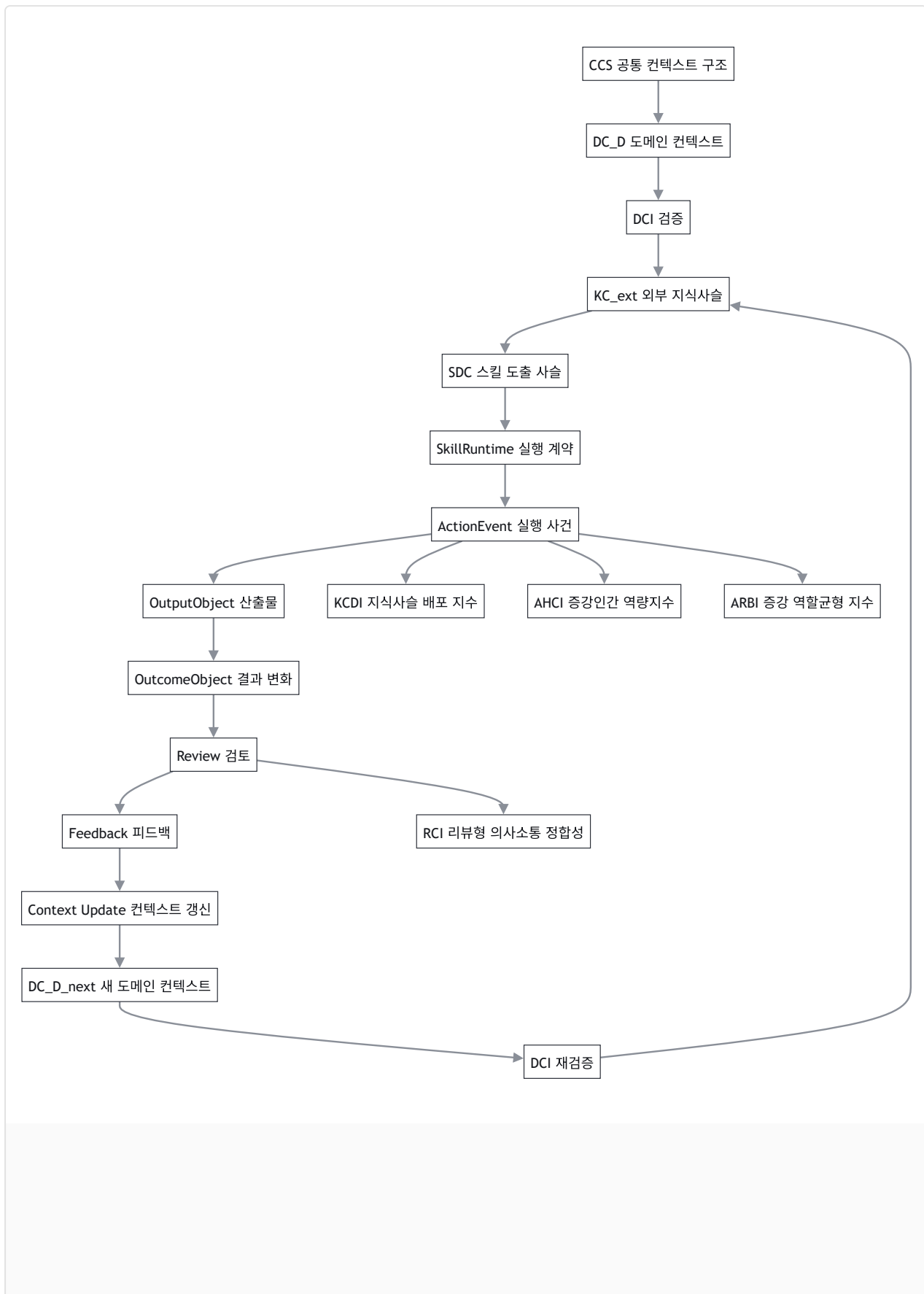


그림 4. KAC와 운영 지표 — ActionEvent에서 KCDI·AHCI·ARBI가, Review에서 RCI가 분기하여 실행·관찰을 측정한다.

11 KAC의 통합 순환 구조

KAC는 직선형 프로세스가 아니라 순환 구조다[R6](그림 5).

```
CCS → DC_D → DCI → Identity 기반 지식사슬 → ValidDerivation → ValidDeployment
→ SkillRuntime → ActionEvent → OutputObject → OutcomeObject
→ KCDI / AHCI / ARBI / RCI → Feedback → DC_D(t+1) → DCI Revalidation
```

그림 5. KAC의 통합 순환 구조 — DCI가 앞단 기준 환경을 검증하고, 지식사슬이 Skill을 도출하며, SkillRuntime·ActionEvent가 실행을 기록하고, 네 지표가 결과를 측정한 뒤 승인된 Feedback이 새 버전을 만들어 다시 DCI 검증으로 닫힌다.

이 순환이 닫힐 때 조직은 단순히 AI를 사용한 것이 아니라, AI 실행 결과를 조직 지식과 기준으로 축적하게 된다.

12 예시 — 고객응대 KAC

KAC를 고객응대 도메인에 적용하면 표 6과 같다.

표 6. 고객응대 도메인의 KAC 적용

KAC 요소	고객응대 예시
<i>KC_ext</i>	제품 정책, 환불 규정, 고객 이력, 법무 기준, FAQ
<i>SDC</i>	CustomerSupportAgent → CustomerIssueResolution → IssueClassification → ProductPolicyKnowledge → ResponseFramingMethod → Policy-grounded Response Skill
<i>SkillRuntime</i>	입력: 고객 문의 / 출처: FAQ·약관 / 가드레일: 법률 자문 금지 / 승인: 환불·보상 건 인간 승인
<i>ActionEvent</i>	AI가 고객 문의를 분류하고 답변 초안을 생성
<i>OutputObject</i>	근거가 연결된 고객 답변 초안
<i>OutcomeObject</i>	문의 해결, 재문의 감소, 정책 위반 없음
<i>Review</i>	RCI로 근거·권한·책임·표현·승인 여부 검토
<i>Feedback</i>	"환불 예외 기준이 모호함"을 변경 후보로 등록
<i>DC_D(t+1)</i>	고객응대 도메인 컨텍스트의 환불 기준 업데이트

여기서 중요한 것은 AI가 답변 초안을 만들었다는 사실이 아니라, 그 답변이 어떤 지식사슬에서 도출된 Skill에 기반했는지, 어떤 Runtime으로 실행되었는지, 실행 사건이 어떻게 기록되었는지, 산출물과 결과가 어떻게 분리되었는지, 검토와 피드백이 어떻게 도메인 컨텍스트를 개선했는지다. 즉 — **AI 답변 생성은 KAC의 일부일 뿐이며, KAC의 본질은 지식이 행동으로 검증되고 그 행동의 결과가 조직 기준을 개선하는 데 있다.**

13 KAC의 의의

- **지식과 행동의 공백을 메운다.** 많은 조직이 지식 저장소·매뉴얼·프롬프트·RAG·에이전트를 별도로 구축하지만 지식이 어떤 Skill로 도출되고 어떤 실행 계약으로 행동이 되는지 설명하지 못한다. KAC는 이를 *Knowledge* → *Skill* → *Runtime* → *Action* → *Outcome*으로 연결한다.

- **Skill을 AX 실행 단위로 재정의한다.** 실제 업무에는 자료 탐색·기준 적용·판단·검증·표현·승인·기록·개선이 필요하므로 실행 단위는 단순 데이터가 아니라 Skill이며[R7], KAC는 그 Skill의 도출·실행·검증을 다룬다.
- **산출물과 결과를 구분한다.** 문서가 만들어졌다고 지식이 축적된 것이 아니다. OutputObject(산출물)와 OutcomeObject(결과 변화)의 구분이 있어야 "AI가 무엇을 만들었는가"가 아니라 "그것이 어떤 변화를 만들었는가"를 평가할 수 있다.
- **Review·Feedback을 실행 사슬에 포함한다.** AI 결과는 검토 없이 조직 기준에 반영되어선 안 되며, RCI가 발화·산출물을 리뷰 객체로 검증하고 승인된 피드백만 컨텍스트 변경 후보가 되게 한다 [R3].
- **AX를 조직 학습 구조로 만든다.** KAC가 없으면 AI 사용은 개별 산출물 생산으로 흩어지지만, KAC가 있으면 실행 결과가 기록·검토·피드백되어 도메인 컨텍스트의 새 버전으로 축적된다.

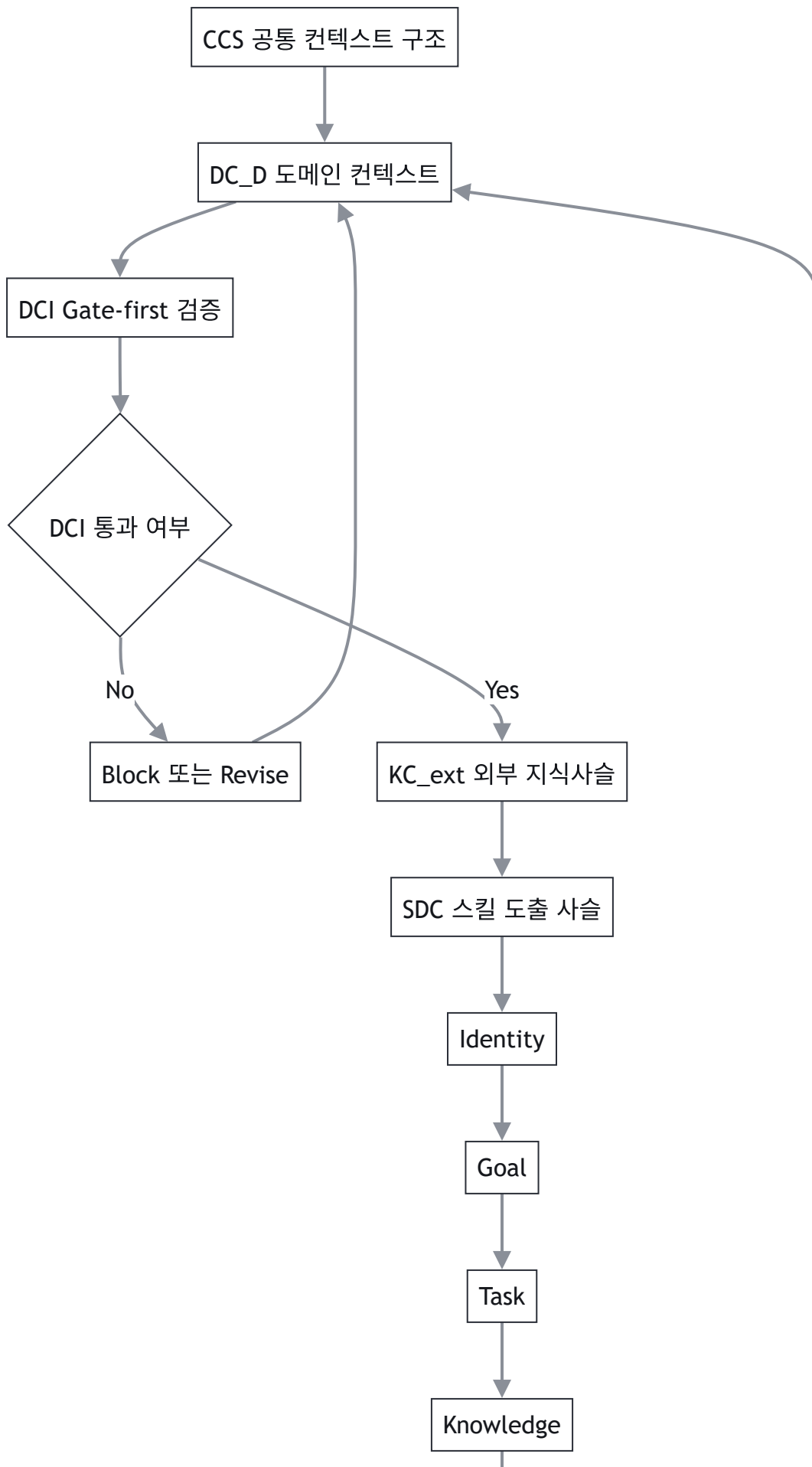
14 결론

AI 시대의 조직 경쟁력은 어떤 AI 모델을 쓰는가만으로 결정되지 않는다. 더 중요한 것은 그 모델이 어떤 공통·도메인 컨텍스트 안에서 작동하는가, 어떤 지식사슬을 통해 Skill을 도출하는가, 그 Skill이 어떤 SkillRuntime으로 실행되는가, 실행 결과가 어떻게 검증되고 조직 기준으로 학습되는가이다. 지식 사슬은 지식의 경로이고, 스킬 도출 사슬은 특정 Identity·Entity가 왜 어떤 Skill을 필요로 하는지 설명하는 추론 경로다. 그러나 조직 AX는 여기서 멈출 수 없다 — 도출된 Skill은 Runtime으로 컴파일되고, ActionEvent로 실행되며, OutputObject와 OutcomeObject를 남기고, Review를 통과해 Feedback으로 정리되어 도메인 컨텍스트를 갱신해야 한다. 이 전체 구조가 KAC다.

KAC가 없으면 지식은 저장된다.
KAC가 있으면 지식은 행동한다.

(13)

더 정확히 말하면 — KAC는 검증된 지식이 실행 가능한 Skill로 도출되고, SkillRuntime을 통해 ActionEvent로 호출되며, Output과 Outcome을 만들고, Review와 Feedback을 거쳐 도메인 컨텍스트를 갱신하는 AX 실행·검증·학습 온톨로지다. 따라서 KAC는 단순한 지식관리 모델이 아니라, AI 시대 조직이 지식을 행동으로 바꾸고, 행동을 결과로 검증하며, 결과를 다시 조직의 기준과 컨텍스트로 축적하기 위한 실행 운영체제다.



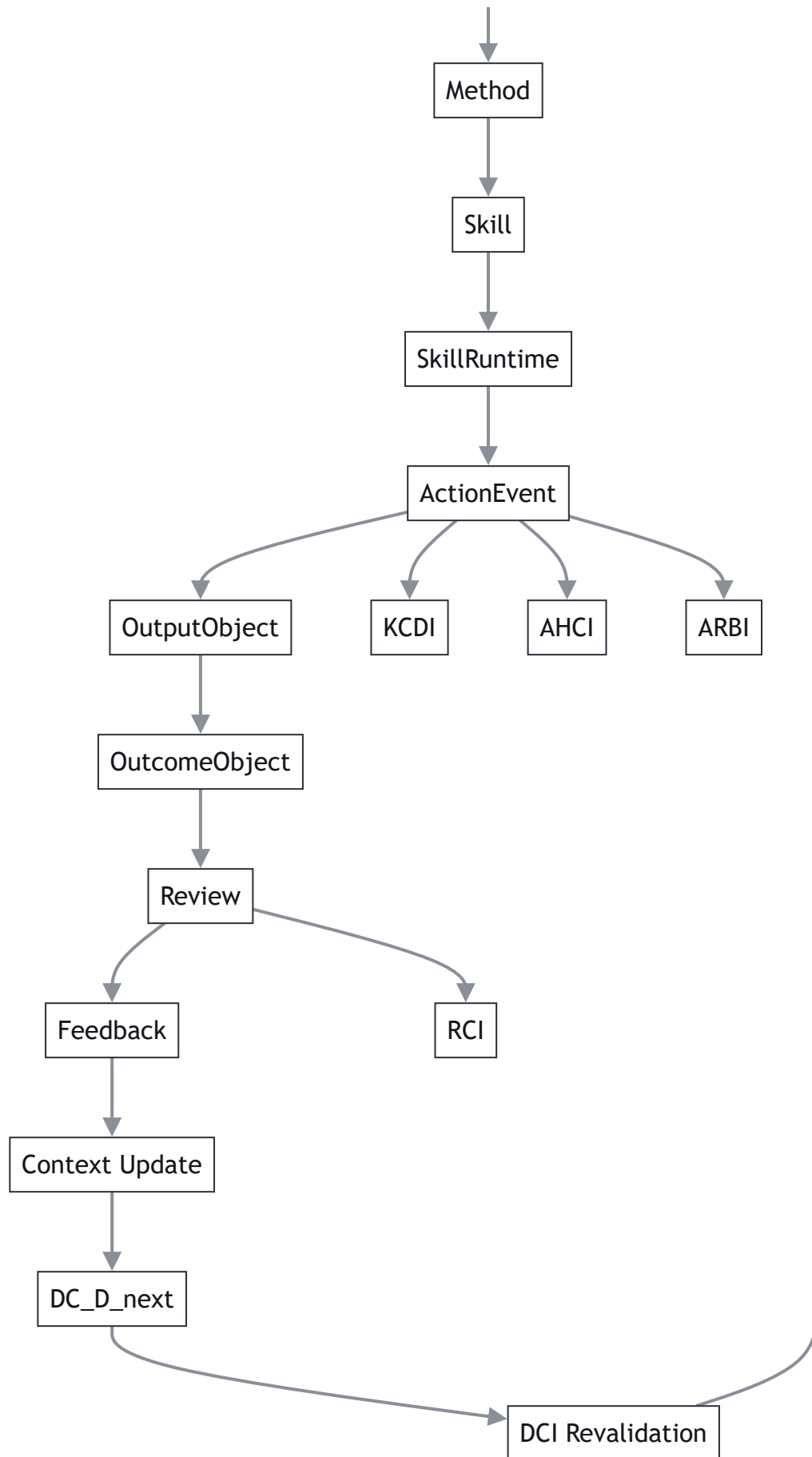


그림 6. 논문용 최종 통합도 — DCI Gate-first 판정(통과/Block·Revise), SDC 전개, 실행 사슬, 그리고 KCDI·AHCI·ARBI·RCI 지표 분기를 하나로 묶은 전체 구조.

저자 기여 및 자료 출처. 본 논문의 KAC 9-튜플·SkillRuntime·ActionEvent/OutputObject/OutcomeObject 분리·ValidKAC 성립 조건·DCI/KCDI/RCI/AHCI/ARBI 연결·통합 순환 구조는 「지식행동 사슬(KAC) 최종글 v1.0」 원고와 지식사슬·KCDI·RCI·DCI·ARBI·AHCI, 공통/거버넌스 컨텍스트, AX 통합 순환 구조 조직 AX 연구자료를 기반으로 정리·체계화하였다. 외부 지식사슬·Knowledge Value Chain·Knowledge-to-Action에 관한 서술은 아래 외부 참고문헌에 근거하며, KAC는 본 체계가 제안하는 응용 실행 온톨로지로서 각 구간의 검증 기준·계수는 실제 도메인 데이터로 보정되어야 하는 운영 파라미터임을 명시한다.

참고문헌

- [R1] 조직 AX 연구자료, 「아이덴티티 기반 스킬 도출 지식사슬의 개발과 배포」, 미간행 원고, 2026.
- [R2] 조직 AX 연구자료, 「지식사슬 배포 지수(KCDI)」, 미간행 원고, 2026.
- [R3] 조직 AX 연구자료, 「리뷰형 의사소통 정합성 지수(RCI v3.0)」, 미간행 원고, 2026.
- [R4] 조직 AX 연구자료, 「공통 컨텍스트 및 거버넌스 컨텍스트」, 미간행 원고, 2026.
- [R5] 조직 AX 연구자료, 「증강 역할균형 지수(ARBI)」 및 「증강인간 역량지수(AHCI)」, 미간행 원고, 2026.
- [R6] 조직 AX 연구자료, 「공통 컨텍스트 기반 AX 실행·검증·학습의 통합 순환 구조」 및 「도메인 컨텍스트 정합성 지수(DCI)」, 미간행 원고, 2026.
- [R7] C. W. Holsapple, M. Singh, "The Knowledge Chain Model: Activities for Competitiveness," in Handbook on Knowledge Management, Springer.
- [R8] IAEA, "Knowledge Management — Knowledge Value Chain." https://gnssn.iaea.org/main/Pages/CapacityBuilding/Knowledge_Management.aspx
- [R9] Registered Nurses' Association of Ontario (RNAO), "Knowledge-to-Action Framework." <https://rnao.ca/bpg/leading-change-toolkit/knowledge-to-action-framework>